Labo 4 – Dataverwerking II

Combineren, samenvatten, herschikken

Cursustekst bij het notebook `lab4_data_processing_II.py`

dr. Maarten Slembrouck

Waarover gaat dit labo?

Vorige week maakte je van rommelige exports nette tabellen: één rij per atleet, één rij per test. Maar de vragen van een coach gaan zelden over één rij. "Hoe groot zijn onze verdedigers gemiddeld?" gaat over een *groep* rijen. "Hoe snel sprinten onze keepers?" heeft informatie uit *twee* tabellen nodig. "Is iedereen vooruitgegaan sinds de eerste test?" vraagt om de tabel anders te *schikken*. En de hartslagdata van een training is een *tijdreeks*, met één meting per seconde.

Na dit labo kun je data per groep samenvatten met `groupby`, tabellen koppelen met `merge`, data herschikken met `melt` en `pivot_table`, en eenvoudige kenmerken van een tijdreeks berekenen (hartslagzones, voortschrijdend gemiddelde, hersampling).

Net als vorige week is elk voorbeeld echt uitgevoerd op de data van de club. Lees deze tekst naast het notebook en typ de voorbeelden zelf over.

1 Vertrekpunt: de opgekuiste data van vorige week

Het notebook begint met de opkuis van labo 3, in compacte vorm. Twee hulpfuncties doen het werk: `clean_positions` vertaalt de 13 schrijfwijzen naar 4 posities, `fix_sprint_units` zet milliseconden om naar seconden.

```
import pandas as pd

def clean_positions(position_column):
    position_map = {
        "forward": "Forward", "fwd": "Forward",
        "midfielder": "Midfielder", "mid": "Midfielder",
        "defender": "Defender", "def": "Defender",
        "goalkeeper": "Goalkeeper", "gk": "Goalkeeper",
    }
    return position_column.str.strip().str.lower().map(position_map)

def fix_sprint_units(seconds_column):
    return seconds_column.where(seconds_column < 20, seconds_column / 1000)

REFERENCE_DATE = pd.Timestamp("2026-09-14")

athletes = pd.read_csv(f"{DATA_DIR}/athletes.csv").assign(
    position=lambda d: clean_positions(d["position"]),
    weight_kg=lambda d: d["weight_kg"].str.replace(",", ".").astype(float),
    age=lambda d: (REFERENCE_DATE - pd.to_datetime(d["birthdate"])).dt.days // 365,
)
tests = pd.read_csv(f"{DATA_DIR}/tests.csv").assign(
```

```
    sprint_10m_s=lambda d: fix_sprint_units(d["sprint_10m_s"]),
    sprint_30m_s=lambda d: fix_sprint_units(d["sprint_30m_s"]),
)
```

Lees deze code rustig: alles erin heb je vorige week gezien. Merk op dat de dubbele atleet hier *bewust* nog niet verwijderd is. Dat wordt belangrijk in sectie 3.

Herhaling: foutmeldingen lezen

Het notebook start met een kapotte cel die je moet herstellen. Een foutmelding ziet er eng uit, maar de belangrijkste informatie staat altijd op de **laatste regel**: het soort fout en een korte uitleg. Twee fouten die je vandaag zeker tegenkomt:

```
athletes["positio"]
```

```
KeyError: 'positio'
```
uitvoer

Een `KeyError` betekent: deze sleutel (hier: kolomnaam) bestaat niet. Bijna altijd een tikfout.

```
pd.Series(["57,5", "60,3"]).astype(float)
```

```
ValueError: could not convert string to float: '57,5'
```
uitvoer

Een `ValueError` bij `astype(float)` betekent: deze tekst kan ik niet omzetten naar een getal. Hier staat de komma in de weg.

2 Groepen samenvatten met `groupby`

2.1 Splitsen, toepassen, combineren

Stel dat de coach de gemiddelde leeftijd per positie wil kennen. In gedachten doe je drie dingen:

1. **splitsen**: de atleten in vier groepjes verdelen, één per positie;
2. **toepassen**: per groepje het gemiddelde berekenen;
3. **combineren**: de vier gemiddelden in één tabel zetten.



In pandas is dat één regel. `groupby("position")` splitst, `["age"]` kiest de kolom, `.mean()` past het gemiddelde toe en combineert:

```
mean_age_per_position = athletes.groupby("position")["age"].mean().round(1)
mean_age_per_position
```

uitvoer
```
position
Defender      14.6
Forward       13.2
Goalkeeper    14.4
Midfielder    14.5
Name: age, dtype: float64
```

Het resultaat is een Series met de posities als *index*. In plaats van `.mean()` kun je ook `.median()`, `.min()`, `.max()`, `.sum()` of `.count()` gebruiken.

`size()` of `count()`?

`.size()` telt het aantal rijen per groep. `.count()` telt het aantal *ingevulde* waarden in een kolom en slaat `NaN` over. Wil je weten hoeveel atleten er per positie zijn, gebruik dan `.size()`.

```
athletes.groupby("position").size()
```

uitvoer
```
position
Defender      12
Forward       14
Goalkeeper     5
Midfielder    12
dtype: int64
```

2.2 Getallen in categorieën: `pd.cut`

Jeugdploegen worden ingedeeld in leeftijdscategorieën: U13, U15, U17, U19. `pd.cut` verdeelt een getallenkolom in vakjes. Je geeft de grenzen (`bins`) en de namen van de vakjes (`labels`) mee:

```
athletes_with_age_group = athletes.assign(
    age_group=lambda d: pd.cut(
        d["age"], bins=[10, 12, 14, 16, 18], labels=["U13", "U15", "U17", "U19"]
    )
)
athletes_with_age_group[["name", "age", "age_group"]].head()
```

uitvoer
```
             name  age age_group
0  Yana De Backer   16       U17
1    Stan Mertens   14       U15
2    Amber Jacobs   16       U17
3    Bram Wouters   17       U19
4  Emma Verhoeven   15       U17
```

Vijf grenzen maken vier vakjes: van 10 tot 12, van 12 tot 14, enzovoort. Een vakje bevat zijn *rechtergrens* maar niet zijn linkergrens: een atleet van 12 valt in U13 (10 tot en met 12), een atleet van 13 in U15. Er is dus altijd één label minder dan er grenzen zijn.

2.3 Groeperen op meerdere kolommen en `agg`

Je kunt op twee kolommen tegelijk groeperen door een lijst mee te geven. Met `.agg(...)` bereken je meerdere statistieken in één keer en geef je elke nieuwe kolom zelf een naam:

```
athletes_with_age_group.groupby(["position", "age_group"], observed=True).agg(
    n_athletes=("athlete_id", "size"),
```

```
    mean_height_cm=("height_cm", "mean"),
)
```

uitvoer

```
                      n_athletes  mean_height_cm
position   age_group
Defender   U13                 2      145.400000
           U15                 5      158.250000
           U17                 3      169.900000
           U19                 2      174.150000
Forward    U13                 7      153.350000
           U15                 4      165.475000
...                          ...             ...
Goalkeeper U15                 1      157.000000
           U17                 3      174.466667
Midfielder U13                 2      157.950000
           U15                 5      164.440000
           U17                 1      163.300000
           U19                 4      171.475000
```

Lees `mean_height_cm=("height_cm", "mean")` als: "maak een kolom `mean_height_cm`, door op de kolom `height_cm` de functie `mean` toe te passen". Een paar extra dingen:

- De index heeft nu twee niveaus (positie en leeftijdscategorie): een *MultiIndex*. Met `.reset_index()` maak je er weer gewone kolommen van.
- `age_group` is een *categorische* kolom (gemaakt door `pd.cut`). `observed=True` zorgt ervoor dat combinaties zonder atleten (bv. keepers in U13) niet als lege rij verschijnen.

Probeer zelf

Bereken de maximale lengte per positie. Groepeer daarna op geslacht en tel met `.size()` hoeveel jongens en meisjes er zijn.

3 Tabellen koppelen met `merge`

3.1 Een sleutel in twee tabellen

`athletes` weet van elke atleet de positie, `tests` kent de sprinttijden. Om de sprinttijd per positie te bekijken, moet je de twee koppelen. Dat kan omdat beide tabellen een kolom `athlete_id` hebben: de **sleutel**.

Een klein voorbeeld maakt duidelijk wat er gebeurt:

```
people = pd.DataFrame({"athlete_id": ["A1", "A2", "A3"],
                       "name": ["Yana", "Stan", "Emma"]})
results = pd.DataFrame({"athlete_id": ["A1", "A1", "A2", "A9"],
                        "sprint_s": [5.3, 5.2, 5.9, 6.1]})
people.merge(results, on="athlete_id", how="inner")
```

uitvoer

```
  athlete_id  name  sprint_s
0         A1  Yana       5.3
1         A1  Yana       5.2
2         A2  Stan       5.9
```

Voor elke rij in `people` zoekt pandas de rijen in `results` met dezelfde `athlete_id`. Yana heeft twee resultaten, dus ze verschijnt twee keer. Emma (A3) heeft geen resultaat en A9 heeft geen

naam: met `how="inner"` vallen ze allebei weg.

```
people.merge(results, on="athlete_id", how="left")
```

uitvoer

```
  athlete_id  name  sprint_s
0         A1  Yana       5.3
1         A1  Yana       5.2
2         A2  Stan       5.9
3         A3  Emma       NaN
```

Met `how="left"` blijft *elke* rij van de linkertabel behouden. Emma staat erin, met `NaN` als sprinttijd.

De soorten joins

inner: alleen sleutels die in beide tabellen staan. **left**: alle rijen van de linkertabel, aangevuld waar mogelijk. **right**: hetzelfde, vanuit de rechtertabel. **outer**: alles uit beide tabellen. Controleer na elke merge het aantal rijen: dat vertelt je of er iets wegviel of verdubbelde.

Met de echte data:

```
merged_inner = athletes.merge(tests, on="athlete_id", how="inner")
merged_left = athletes.merge(tests, on="athlete_id", how="left")
(len(athletes), len(tests), len(merged_inner), len(merged_left))
```

uitvoer

```
(43, 114, 114, 114)
```

Hier zijn beide even lang, omdat elke atleet minstens één testresultaat heeft. Met echte data is dat zelden gegarandeerd.

3.2 De dubbele atleet

Eén atleet staat twee keer in `athletes`, met twee verschillende ID's. Je vindt die rijen met `duplicated`; met `keep=False` worden *alle* exemplaren gemarkeerd, niet alleen de tweede:

```
same_person = athletes.duplicated(subset=["name", "birthdate"], keep=False)
athletes.loc[same_person, ["athlete_id", "name", "birthdate"]]
```

uitvoer

```
   athlete_id       name   birthdate
10       A011  Liam Maes  2014-10-20
42       A043  Liam Maes  2014-10-20
```

De merge doet perfect wat gevraagd is, maar weet niet dat het om dezelfde persoon gaat. Na de merge staan de testen van die atleet onder twee ID's:

```
dupe_name = athletes.loc[same_person, "name"].iloc[0]
merged_inner.loc[merged_inner["name"] == dupe_name,
                 ["athlete_id", "name", "test_moment", "sprint_30m_s"]]
```

uitvoer

```
    athlete_id       name  test_moment  sprint_30m_s
24        A011  Liam Maes            1          5.35
25        A011  Liam Maes            2          5.29
26        A011  Liam Maes            3          5.37
111       A043  Liam Maes            1          4.64
112       A043  Liam Maes            2          4.59
113       A043  Liam Maes            3          4.46
```

Elke statistiek die je na deze merge berekent, telt deze atleet dubbel. De oplossing: verwijder de dubbels *vóór* de merge, op naam en geboortedatum. Na de merge kan dat niet meer, want dan delen ook de verschillende testmomenten van één atleet dezelfde naam en geboortedatum: je zou echte testresultaten weggooien.

```
athletes_unique = athletes.drop_duplicates(subset=["name", "birthdate"],
↪  keep="first")
combined = athletes_unique.merge(tests, on="athlete_id", how="inner")
combined.shape
```

uitvoer

```
(111, 15)
```

3.3 Verzamelingen en `isin`

Om te weten welke rijen bij het ontdubbelen verdwenen, heb je twee hulpmiddelen nodig.

Een **verzameling** (`set`) is een lijst zonder dubbels en zonder volgorde. Het verschil van twee verzamelingen (`a - b`) geeft de elementen die in `a` zitten maar niet in `b`:

```
set(["A1", "A2", "A3"]) - set(["A2"])
```

uitvoer

```
{'A3', 'A1'}
```

`.isin(...)` geeft een masker dat `True` is voor elke rij waarvan de waarde in de opgegeven lijst of verzameling staat:

```
results["athlete_id"].isin({"A1", "A9"})
```

uitvoer

```
0     True
1     True
2    False
3     True
Name: athlete_id, dtype: bool
```

Met deze twee bouwstenen (en `.sum()` op een masker) kun je oefening 2 in het notebook oplossen.

4 Lang en breed: `pivot_table` en `melt`

4.1 Twee vormen voor dezelfde data

`combined` is **lang**: één rij per atleet *per testmoment*. Een atleet met drie testen staat er drie keer in. Dezelfde informatie kan ook **breed**: één rij per atleet, met een kolom per testmoment.

```
sprint_wide = combined.pivot_table(
    index="athlete_id", columns="test_moment", values="sprint_30m_s"
)
sprint_wide.head()
```

uitvoer

```
test_moment     1     2     3
athlete_id
A001         5.34  5.32   NaN
A002         5.97  5.74   NaN
A003         5.86  5.87   NaN
A004         5.33  5.25  5.28
A005         5.59  5.66   NaN
```

De drie argumenten lees je zo:

- `index`: wat wordt een rij (één per atleet);
- `columns`: welke waarden worden kolommen (de testmomenten 1, 2 en 3);
- `values`: welke waarde komt in de cellen (de sprinttijd).

Ontbreekt een combinatie, dan krijg je `NaN`. Zijn er meerdere rijen voor dezelfde combinatie, dan neemt `pivot_table` standaard het gemiddelde.

4.2 Waarom breed handig is

In de brede vorm is vooruitgang tussen twee testmomenten één aftrekking. Let op: de kolomnamen zijn hier *getallen*, geen tekst. Je schrijft dus `sprint_wide[2]`, niet `sprint_wide["2"]`.

```
sprint_progress_1_to_2 = sprint_wide[2] - sprint_wide[1]
sprint_progress_1_to_2.describe()
```

uitvoer

```
count    41.000000
mean     -0.069512
std       0.110339
min      -0.270000
25%      -0.170000
50%      -0.070000
75%       0.010000
max       0.140000
dtype: float64
```

Een negatief getal betekent hier vooruitgang: een lagere sprinttijd is sneller. Bij spronghoogte is het omgekeerd: daar is hoger beter.

4.3 Terug naar lang: `melt`

`melt` doet het omgekeerde van `pivot_table`: kolommen worden weer rijen. Plotly express (labo 5) werkt het liefst met lange data. Eerst maak je met `.reset_index()` van de index weer een gewone kolom:

```
sprint_long_again = sprint_wide.reset_index().melt(
    id_vars="athlete_id", var_name="test_moment", value_name="sprint_30m_s"
)
sprint_long_again.head()
```

uitvoer

```
  athlete_id test_moment  sprint_30m_s
0       A001           1          5.34
1       A002           1          5.97
2       A003           1          5.86
3       A004           1          5.33
4       A005           1          5.59
```

`id_vars` zijn de kolommen die blijven staan, `var_name` is de naam van de nieuwe kolom met de vroegere kolomnamen, `value_name` de naam van de kolom met de waarden.

5 Tijdreeksen: de hartslag tijdens een training

`hr_session.csv` bevat per seconde de hartslag en snelheid van vijf atleten tijdens één training. Een tabel waarin elke rij een moment in de tijd is, heet een **tijdreeks**.

5.1 Een tijdstempel maken

Het bestand heeft een datum en het aantal seconden sinds de start (`time_s`). Samen maken ze een tijdstempel. `pd.to_timedelta(..., unit="s")` zet een aantal seconden om in een tijdsduur, die je bij een datum kunt optellen:

```
hr_session = pd.read_csv(f"{DATA_DIR}/hr_session.csv")
hr_session["timestamp"] = pd.to_datetime(hr_session["date"]) + pd.to_timedelta(
    hr_session["time_s"], unit="s"
)
hr_session.head(3)
```

uitvoer

	athlete_id	date	time_s	heart_rate_bpm	speed_kmh	timestamp
0	A001	2026-11-09	0	85.0	0.94	2026-11-09 00:00:00
1	A001	2026-11-09	1	80.0	2.84	2026-11-09 00:00:01
2	A001	2026-11-09	2	80.0	2.58	2026-11-09 00:00:02

5.2 Hartslagzones

Trainers drukken intensiteit uit als percentage van de *maximale* hartslag. Een veelgebruikte vuistregel schat die als 220 min de leeftijd. De leeftijd staat in `athletes`, dus eerst een merge:

```
ages = athletes[["athlete_id", "age"]]
hr_with_age = hr_session.merge(ages, on="athlete_id", how="left")
hr_with_age["hr_max"] = 220 - hr_with_age["age"]
hr_with_age["pct_hr_max"] = (
    100 * hr_with_age["heart_rate_bpm"] / hr_with_age["hr_max"]
)
hr_with_age[["athlete_id", "heart_rate_bpm", "hr_max", "pct_hr_max"]].head(3)
```

uitvoer

	athlete_id	heart_rate_bpm	hr_max	pct_hr_max
0	A001	85.0	204	41.666667
1	A001	80.0	204	39.215686
2	A001	80.0	204	39.215686

Met `pd.cut` deel je het percentage in vijf zones in. Het merkwaardige getal 1000 als laatste grens zorgt ervoor dat ook een percentage boven 100 nog in zone 5 valt:

```
hr_zoned = hr_with_age.assign(
    hr_zone=lambda d: pd.cut(
        d["pct_hr_max"],
        bins=[0, 60, 70, 80, 90, 1000],
        labels=["Zone 1", "Zone 2", "Zone 3", "Zone 4", "Zone 5"],
    )
)
hr_zoned["hr_zone"].value_counts()
```

uitvoer

```
hr_zone
Zone 4    6172
Zone 1    2344
```

```
Zone 3    1517
Zone 5    1369
Zone 2     573
Name: count, dtype: int64
```

uitvoer

5.3 Tijd per zone

Elke rij is één seconde. Het aantal rijen per atleet en per zone, gedeeld door 60, geeft dus het aantal minuten:

```
time_per_zone_minutes = (
    hr_zoned.groupby(["athlete_id", "hr_zone"], observed=True).size() / 60
).rename("minutes").reset_index()
time_per_zone_minutes.head(5)
```

uitvoer

	athlete_id	hr_zone	minutes
0	A001	Zone 1	8.350000
1	A001	Zone 2	1.583333
2	A001	Zone 3	7.883333
3	A001	Zone 4	19.400000
4	A001	Zone 5	2.683333

Wil je enkel bepaalde zones, filter dan eerst de rijen met `.isin([...])` en groepeer daarna. Dat is de kern van oefening 4.

5.4 Voortschrijdend gemiddelde en hersampling

Een hartslag per seconde springt op en neer. Twee technieken maken de reeks rustiger. Beide hebben een tijdreeks nodig die *gesorteerd* is op tijd, met de tijd als index (`set_index("timestamp")`):

```
one_athlete = hr_zoned.loc[hr_zoned["athlete_id"] == "A001"]
one_athlete = one_athlete.sort_values("timestamp")
hr_series = one_athlete.set_index("timestamp")["heart_rate_bpm"]
```

Een **voortschrijdend gemiddelde** (`rolling`) berekent op elk moment het gemiddelde van de voorbije 30 seconden. Je houdt even veel rijen, maar de lijn wordt vloeiender:

```
hr_series.rolling("30s").mean().iloc[[0, 1, 29, 30, 31]]
```

uitvoer

```
timestamp
2026-11-09 00:00:00    85.000000
2026-11-09 00:00:01    82.500000
2026-11-09 00:00:29    81.833333
2026-11-09 00:00:30    81.800000
2026-11-09 00:00:31    81.933333
Name: heart_rate_bpm, dtype: float64
```

Hersampling (`resample`) groepeert de tijdreeks in vaste blokken, hier van één minuut, en vat elk blok samen. Je krijgt veel minder rijen: één per minuut.

```
hr_series.resample("1min").mean().head(4)
```

uitvoer

```
timestamp
2026-11-09 00:00:00     82.150000
2026-11-09 00:01:00     85.000000
2026-11-09 00:02:00     99.933333
```

```
2026-11-09 00:03:00    110.466667
Freq: min
Name: heart_rate_bpm, dtype: float64
```
uitvoer

`rolling` of `resample`?

`rolling` geeft een waarde per oorspronkelijke rij (een glijdend venster); handig om een grafiek vloeiender te maken. `resample` geeft een waarde per tijdsblok; handig om per minuut of per kwartier samen te vatten. Je kunt `resample` zien als een `groupby` op tijd.

6 De minicase: een trainingsrapport

In de minicase (per twee) maak je één rij per atleet met de minuten in zone 4–5, de maximale snelheid en de afgelegde afstand. Twee ideeën helpen je daarbij.

Afstand uit snelheid. De snelheid staat in km/u en er is één meting per seconde. In één seconde leg je dus `speed_kmh / 3600` kilometer af. De som van die afstanden over de hele training is de totale afstand.

Series combineren via de index. Als twee Series dezelfde index hebben (bv. de `athlete_id`), plaatst pandas de waarden bij een toewijzing of berekening automatisch naast de juiste rij, ook al is de volgorde anders. Dat heet *uitlijnen op de index*:

```
report = pd.DataFrame({"max_speed": [27.1, 24.5]}, index=["A002", "A001"])
extra = pd.Series([10.0, 12.5], index=["A001", "A002"])
report["minutes"] = extra
report
```

```
      max_speed  minutes
A002       27.1     12.5
A001       24.5     10.0
```
uitvoer

A001 krijgt 10.0 en A002 krijgt 12.5, ook al stonden ze in `extra` in een andere volgorde dan in `report`.

7 Samenvatting

Code	Wat het doet
`df.groupby("a")["b"].mean()`	gemiddelde van `b` per waarde van `a`
`df.groupby("a").size()`	aantal rijen per groep
`df.groupby(["a", "b"]).agg(naam=("kol", "mean"))`	meerdere benoemde statistieken per groep
`pd.cut(kol, bins=..., labels=...)`	getallen indelen in categorieën
`.reset_index()`	index terug omzetten naar gewone kolommen
`links.merge(rechts, on="sleutel", how="inner")`	twee tabellen koppelen op een sleutel
`set(a) - set(b)`, `kol.isin(lijst)`	verschil van verzamelingen, masker “zit in”
`df.pivot_table(index=, columns=, values=)`	van lang naar breed
`df.melt(id_vars=, var_name=, value_name=)`	van breed naar lang
`pd.to_timedelta(kol, unit="s")`	aantal seconden naar tijdsduur
`s.rolling("30s").mean()`	voortschrijdend gemiddelde over 30 seconden
`s.resample("1min").mean()`	gemiddelde per blok van één minuut

Veelgemaakte fouten

- Na een merge het aantal rijen niet controleren.
- Dubbels verwijderen *na* de merge in plaats van ervoor.
- `.count()` gebruiken om rijen te tellen terwijl er ontbrekende waarden zijn; gebruik `.size()`.
- Kolomnamen na een `pivot_table` als tekst schrijven (`wide["2"]`) terwijl het getallen zijn (`wide[2]`).
- `rolling` of `resample` gebruiken op data die niet op tijd gesorteerd is, of zonder tijdsindex.

Begrippen

groupby	splitsen in groepen, per groep iets berekenen, en de resultaten combineren.
aggregatie	een groep waarden samenvatten tot één waarde (gemiddelde, som, aantal).
MultiIndex	index met meerdere niveaus, bv. positie en leeftijdscategorie.
sleutel	kolom waarmee je twee tabellen koppelt.
join	manier van koppelen: inner, left, right of outer.
lang / breed	één rij per meting, of één rij per atleet met een kolom per meting.
tijdreeks	tabel waarin elke rij een moment in de tijd is.